



**SECRETARIA DE EDUCACIÓN PÚBLICA
TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE VERACRUZ**

ASIGNATURA:

BD NOSQL.

CARRERA:

ING. SISTEMAS COMPUTACIONALES.

HORA:

03-04 P.M.

ALUMNO:

RAYON HERNANDEZ RAFAEL. - 22020821

TEMA:

DESPLIEGUE Y GESTIÓN LOCAL DE MONGODB MEDIANTE
PODMAN.

SEMESTRE:

AGOSTO - DICIEMBRE 2026.

VERACRUZ, VER. A 23 DE SEPTIEMBRE DE 2026.



Introducción

En este reporte explico paso a paso lo que hice para dejar funcionando una base de datos MongoDB en mi propia computadora con Windows, usando Podman en lugar de instalarla directo como cualquier programa.

Primero tuve que preparar el equipo activando la virtualización con WSL2, luego instalé Podman, y al final levanté MongoDB dentro de lo que se llama un contenedor (que básicamente es un programa empaquetado con todo lo que necesita para funcionar, corriendo aislado del resto de mi PC).

El objetivo de todo esto fue comprobar que usar contenedores es mucho más práctico y ligero que crear una máquina virtual completa, aprender a encender y apagar el servicio cuando lo necesito, y entender qué pasa con mis datos cuando cierro o borro el contenedor.

Preparación del entorno: activar la virtualización e instalar Podman

Para que Podman pueda correr programas de Linux dentro de Windows, primero hay que dejar lista una "base" que permite la virtualización. Son dos pasos.

Activar la virtualización (WSL2)

WSL2 (Windows Subsystem for Linux) es una función de Windows que deja correr cosas de Linux sin tener que instalar otro sistema operativo aparte. Podman la usa para correr los contenedores de forma aislada y sin comer mucha RAM.

Lo que hice fue:

1. Revisé que mi procesador tuviera la virtualización activada (eso se ve en la BIOS/UEFI al encender el equipo).
2. Abrí la terminal de Windows **como administrador** y escribí el comando que instala WSL2: `wsl --install`
3. Reinicié la computadora para que los cambios tomaran efecto.

Descargar e instalar Podman

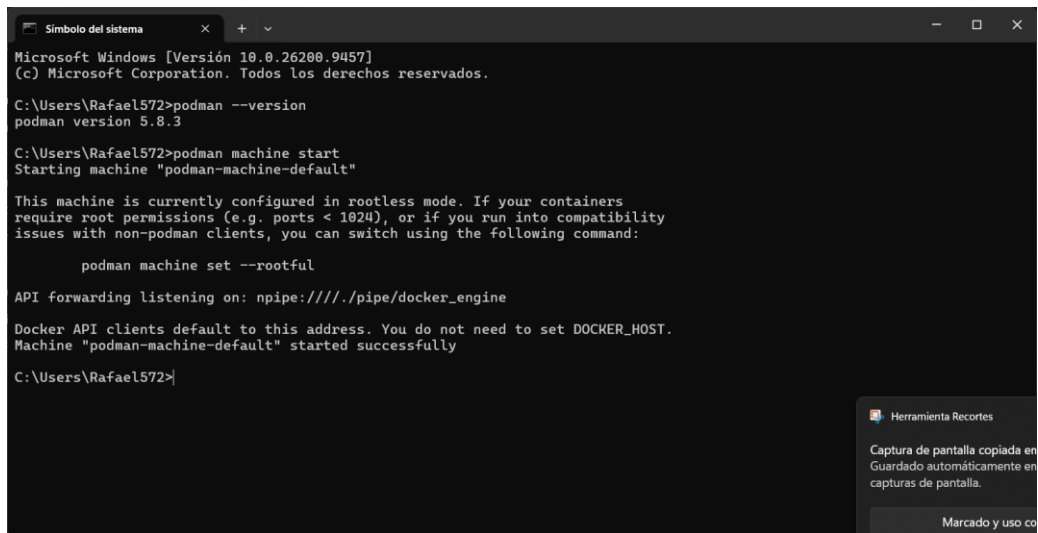
Con WSL2 listo, ya podía instalar Podman:

1. Instalé Podman desde el gestor de paquetes de Windows (**winget**), que es como una "tienda de aplicaciones" pero desde la terminal: `winget install RedHat.Podman`
2. Después de instalarlo, hay que crear y encender la "máquina" interna de Podman, que es donde en realidad van a correr los contenedores:

```
podman machine init
```

```
podman machine start
```

3. Para confirmar que todo quedó bien, pedí la versión instalada:



```
Símbolo del sistema
Microsoft Windows [Versión 10.0.26200.9457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Rafael572>podman --version
podman version 5.8.3

C:\Users\Rafael572>podman machine start
Starting machine "podman-machine-default"

This machine is currently configured in rootless mode. If your containers
require root permissions (e.g. ports < 1024), or if you run into compatibility
issues with non-podman clients, you can switch using the following command:

    podman machine set --rootful

API forwarding listening on: npipe:///./pipe/docker_engine

Docker API clients default to this address. You do not need to set DOCKER_HOST.
Machine "podman-machine-default" started successfully

C:\Users\Rafael572>
```

Herramienta Recortes
Captura de pantalla copiada en
Guardado automáticamente en
capturas de pantalla.
Marcado y uso co

¿Por qué usar Podman y no una máquina virtual tradicional?

Antes de decidirme, comparé dos formas de levantar MongoDB en mi equipo:

1. **Opción 1: Podman sobre WSL2** (la que implementé).
2. **Opción 2: Una máquina virtual normal** (Hyper-V o VirtualBox instalando un Windows/Linux desde un archivo ISO).

Tabla comparativa

Qué comparamos	Opción 1: Podman (WSL2)	Opción 2: Máquina virtual
Memoria RAM	Se usa solo lo necesario. Toma RAM mientras MongoDB trabaja y la suelta cuando no.	Se aparta un bloque fijo. Reserva 4 u 8 GB desde el inicio, aunque no estés haciendo nada.
Tiempo de encendido	Muy rápido: el servicio arranca en 2 a 5 segundos.	Lento: tiene que "prender" un sistema operativo completo (30 s a 2 min).
Conexión de red	Sencilla: me conecto directo desde localhost:27017 , igual que si estuviera instalado normal.	Complicada: la VM tiene su propia IP y hay que configurar un puente de red para llegar a ella.
Moverlo o copiarlo	Fácil: con los mismos comandos funciona en cualquier sistema operativo.	Pesado: hay que exportar archivos de disco gigantes (.ova, vhdx).

En resumen: la máquina virtual es como rentar un departamento entero para guardar una caja, y Podman es como usar un cajón de tu propio escritorio.

Puesta en marcha de MongoDB

Crear el contenedor de MongoDB

Para tener la base de datos corriendo usé este comando en la terminal (CMD):

```
podman run --detach --name todoDB -p 27017:27017 docker.io/mongodb/mongodb-community-server: latest.
```

¿Qué significa cada parte?

- **run**: le dice a Podman que arranque un contenedor. Si no tengo la imagen descargada, la baja automáticamente.
- **--detach**: lo corre "de fondo", así sigo usando la terminal sin que se quede bloqueada.
- **--name todoDB**: le pongo un nombre para no tener que recordar un código largo al referirme a él.
- **-p 27017:27017**: conecta el puerto de mi computadora con el puerto de adentro del contenedor. El 27017 es el puerto por defecto de MongoDB.
- **docker.io/mongodb/mongodb-community-server: latest**: de dónde se descarga el programa (la imagen oficial de MongoDB, en su versión más reciente).

Encender y apagar el contenedor

Para el uso diario solo necesito estos comandos:

- **Cuando voy a trabajar** (encender todo):

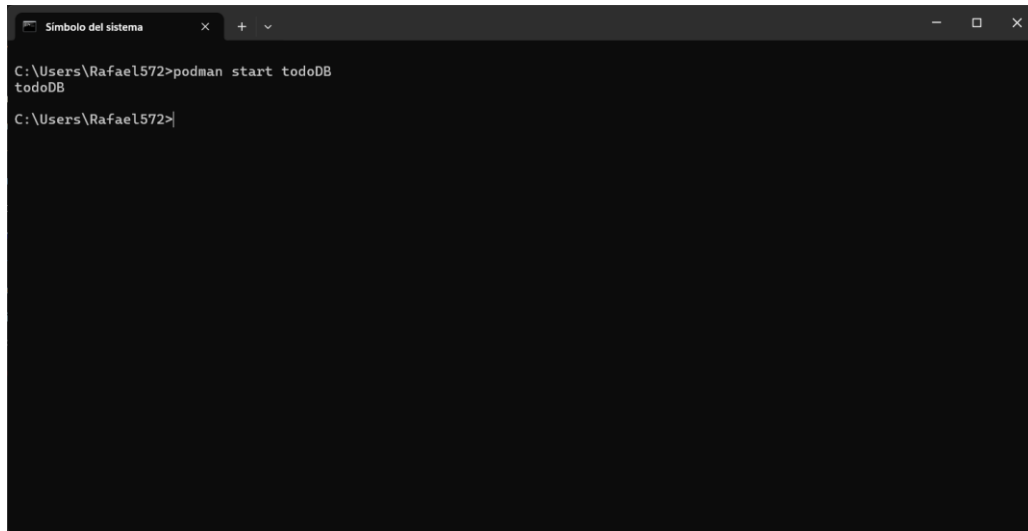
```
podman machine start
```

```
podman start todoDB.
```

- **Cuando termino** (apagar y liberar memoria):

```
podman stop todoDB.
```

```
podman machine stop
```



```
Símbolo del sistema
C:\Users\Rafael572>podman start todoDB
todoDB
C:\Users\Rafael572>|
```

Entrar a la base de datos y hacer consultas (mongosh)

Para "hablarle" a la base de datos usé la consola de MongoDB (llamada **mongosh**) que ya viene adentro del contenedor. Así entro:

(**exec** significa "ejecuta algo dentro de ese contenedor", e **-it** es para que se abra en modo interactivo y pueda escribir comandos.)

Ya dentro, practiqué estas operaciones:

1. **Ver qué bases de datos hay:**

```
show dbs
```

2. **Entrar a una base de datos (si no existe, la crea):**

```
use mi_tienda
```

3. **Ver las colecciones** (que son como las "tablas" de una base de datos relacional):

```
Showcollections
```

4. **Ver los documentos guardados de forma ordenada:**

```
db.productos.find().pretty()
```

```

mongosh mongodb://127.0.0.1:27017
ee http://dochub.mongodb.org/core/prodnotes-filesystem
2026-09-23T23:05:08.631+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-09-23T23:05:08.632+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-09-23T23:05:08.632+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-09-23T23:05:08.632+00:00: We suggest setting the contents of sysfsFile to 0.
2026-09-23T23:05:08.632+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.

test> show dbs
admin      48.00 KiB
config     72.00 KiB
local      72.00 KiB
mi_tienda  48.00 KiB
test> use mi_tienda
switched to db mi_tienda
mi_tienda> show collections
productos
mi_tienda> db.productos.find().pretty()
[
  {
    _id: ObjectId('64b1c30f9c36791e058ce5b0'),
    nombre: 'Laptop',
    precio: 1200,
    enStock: true
  }
]
mi_tienda>

```

¿Qué pasa con mis datos cuando apago todo?

Datos que se borran vs. datos que se quedan

Hay bases de datos que guardan todo en la **memoria RAM** para ser rapidísimas, pero si las apagas... se borra todo, como cuando cierras un juego sin guardar.

Con los contenedores pasa algo parecido por defecto: los datos se guardan en una especie de "borrador" que vive solo mientras el contenedor existe. Si yo borro el contenedor con **podman rm**, **me llevo por delante también los datos**.

Cómo revisé esto en mi contenedor

Para ver si mis datos están seguros fuera del contenedor, usé:

```
podman inspect todoDB
```

Ese comando muestra toda la información del contenedor en formato JSON. Busqué la sección llamada **"Mounts"** (que significa "discos conectados") y me di cuenta de que no tenía ninguno conectado: todo estaba adentro del contenedor.

Para que los datos se guarden de verdad en mi disco duro y sobrevivan, aunque borre el contenedor, hay que conectarle una carpeta de Windows con el parámetro **-v**:

```
podman run --detach --name todoDB -p 27017:27017 -v
C:\RutaLocal\mongo_data:/data/db: Z docker.io/mongodb/mongodb-community-
server: latest.
```

Así la carpeta **C:\RutaLocal\mongo_data** de mi PC queda pegada a la carpeta **/data/db** donde MongoDB guarda todo por dentro.

Conclusión

Con esta actividad logré dejar funcionando MongoDB en mi computadora de forma rápida y sin sacrificar recursos. Podman con WSL2 resultó ser mucho más práctico que una máquina virtual: arranca en segundos, usa solo la RAM que necesita y no me complica la conexión a la base de datos.

También aprendí algo que considero lo más importante del ejercicio: **crear el contenedor no es suficiente, hay que pensar en dónde se guardan los datos.**

Sin un volumen conectado, toda la información se pierde al borrar el contenedor. Saber esto marca la diferencia entre un proyecto de práctica y un proyecto que de verdad puede funcionar en un entorno real.